CSE 4224: Digital System Design Laboratory

Smart Elevator Control System Using FPGA

Submitted By

Sayma Mushsharat

Roll: 2007036

Department of Computer Science and Engineering
Khulna University of Engineering & Technology
Khulna 9203, Bangladesh

Contents

1 Introduction **2**

2 Objectives **2**

3 System Overview **3**
3.1 High-Level Block Diagram 3
3.2 Passenger Entry Workflow 4
3.3 Sweep Processing and Payment Workflow 4

4 System Architecture **5**
4.1 Detailed Architecture Diagram 5
4.2 Module Hierarchy 9
4.3 RTL Schematic 10

5 Data Flow **10**
5.1 Key Signal Descriptions 11

6 Module Descriptions **11**
6.1 Control Unit (FSM) — `control_unit.v` 11
6.2 Fare ALU — `alu.v` 12
6.3 Passenger Memory — `memory_unit.v` 13
6.4 Clock Divider — `clk_divider.v` 13
6.5 Seven-Segment Display — `seven_segment.v` 13
6.6 Button Edge Detector — `btn_edge.v` 14

7 Implementation Details **14**
7.1 Basys3 FPGA Specifications 14
7.2 Input/Output Pin Mapping 15
7.3 Fare Calculation 15

8 Testing and Verification **16**

9 Results and Discussion **16**

10 Conclusion **17**

11 Future Work **17**

1 Introduction

Modern multi-storey buildings require efficient vertical transportation systems that minimise passenger wait time, optimise energy consumption, and enforce a transparent fare structure. Traditional elevator controllers rely on microcontroller-based firmware subject to operating-system scheduling jitter, making it difficult to guarantee deterministic real-time behaviour. Embedding the complete control logic inside a Field-Programmable Gate Array (FPGA) yields a fully parallel, clock-synchronous implementation where every state transition, display update, and I/O sample occurs at a predictable point on the 100 MHz system clock edge—properties that are inherently difficult to achieve in software-only designs.

FPGAs are reconfigurable digital devices composed of look-up tables (LUTs), flip-flops, block RAM, DSP slices, and programmable interconnect. They are described using Hardware Description Languages (HDLs) such as Verilog or VHDL and synthesised into gate-level netlists by tools such as Xilinx Vivado. Unlike microcontrollers, which execute sequential instructions, FPGAs implement parallel combinational and sequential logic operating on every rising clock edge simultaneously. This parallelism makes them well-suited to real-time I/O-intensive control tasks such as display multiplexing, button debouncing, concurrent passenger tracking, and sweep-based scheduling.

This project implements a complete smart elevator control system on the Digilent Basys3 board (Xilinx Artix-7 XC7A35T). The system manages an eight-floor building with up to four passengers queued simultaneously. Passengers are added via debounced button presses with floor selections on slide switches. A sweep-based FSM picks up all passengers at their respective start floors and drops them off at their destinations during a single directional sweep—just like a real elevator. Upon arrival, a fare of 5 taka per floor is displayed on a multiplexed seven-segment display, and payment is confirmed before the next passenger is served. Direction LEDs indicate upward or downward movement, and a memory-full LED warns when the four-passenger queue is saturated. All logic runs off the on-board 100 MHz oscillator, with a 2 Hz tick divider applied for realistic floor-by-floor movement.

2 Objectives

The specific technical objectives established for this project are:

1. Design and implement a modular, fully synchronous Verilog system capable of controlling an eight-floor elevator serving up to four passengers per batch.
2. Implement a nine-state sweep-based FSM that picks up all passengers at their start floors and drops them off at their destinations during a single directional sweep, handling automatic direction determination, pickup, and dropoff detection using parallel combinational comparators.
3. Develop a combinational ALU that computes the absolute floor difference $|\text{dest} - \text{start}|$ for fare calculation, meeting 100 MHz timing closure without pipelining.
4. Implement a four-entry FIFO passenger memory with guarded writes that reject same-floor bookings (start = dest) and prevent overflow beyond four passengers.

5. Construct a two-digit multiplexed seven-segment display driver supporting four display modes: passenger count (IDLE), current floor (movement), fare in taka (payment), and completion indicator "EE" (DONE).
6. Debounce all push-button inputs using a synchronous two-flip-flop metastability barrier and a 20-bit saturating counter providing approximately 10 ms of debounce window, followed by a rising-edge detector for single-cycle pulse generation.
7. Generate two timing domains from the 100 MHz system clock: a 2 Hz slow tick for floor-by-floor elevator movement and a 1 kHz segment tick for flicker-free display multiplexing.
8. Map all I/O to the correct physical Basys3 pins through an XDC constraint file, verify timing closure in Vivado, and validate operation on physical hardware.

3 System Overview

3.1 High-Level Block Diagram

Figure 1 presents the high-level block diagram of the FPGA Smart Elevator Control System. User inputs (three push buttons and sixteen slide switches) feed into the main Elevator Controller block. The controller outputs drive a two-digit multiplexed seven-segment display and three status LEDs. A synchronous reset clears all internal state at any time.

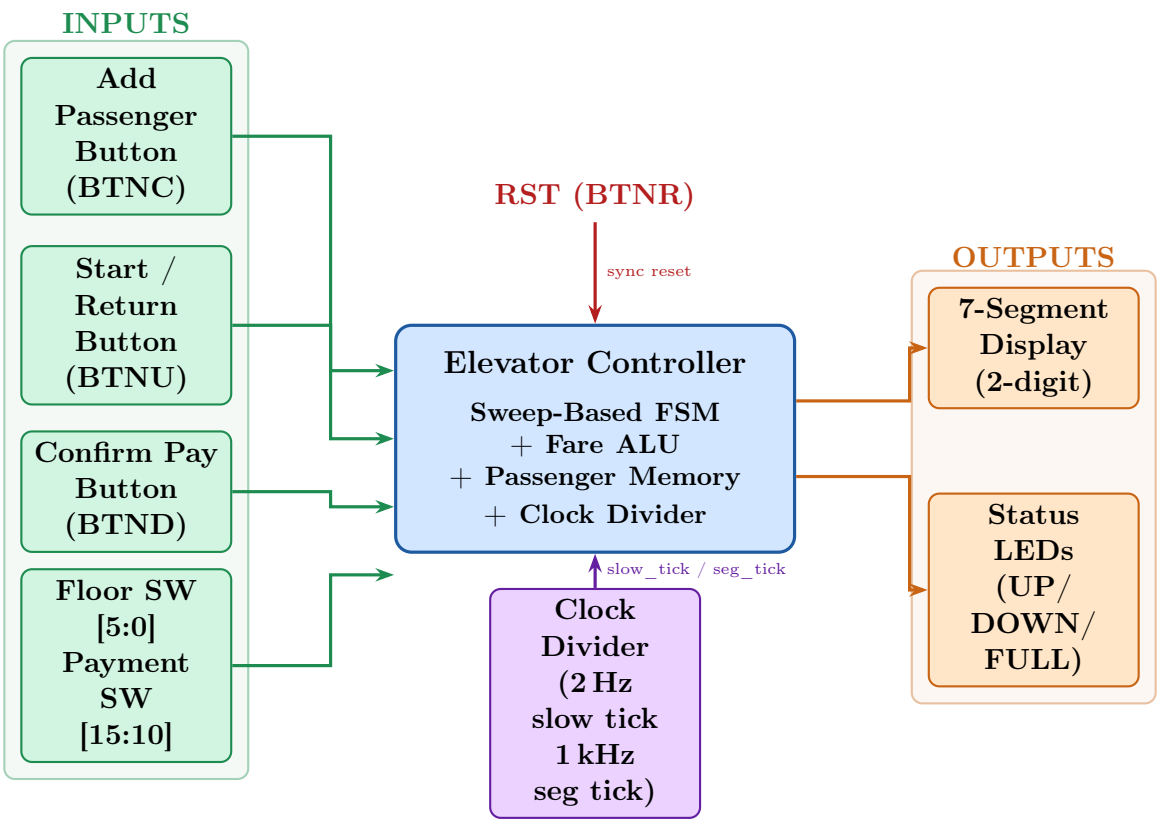

Figure 1: High-Level Block Diagram — FPGA Smart Elevator Control System

3.2 Passenger Entry Workflow

- The user sets `SW[5:3]` to the desired start floor (0–7) and `SW[2:0]` to the destination floor (0–7).
- The user presses BTNC (centre button). The debounce module suppresses contact bounce for approximately 10 ms and delivers a guaranteed single-cycle rising-edge pulse to the FSM.
- The memory unit validates the request: bookings with start = dest are silently rejected, and a full queue (count = 4) prevents further writes. On success, the passenger count on the seven-segment display increments.
- The user may repeat this process for up to four passengers before pressing BTNU (up button) to start processing.

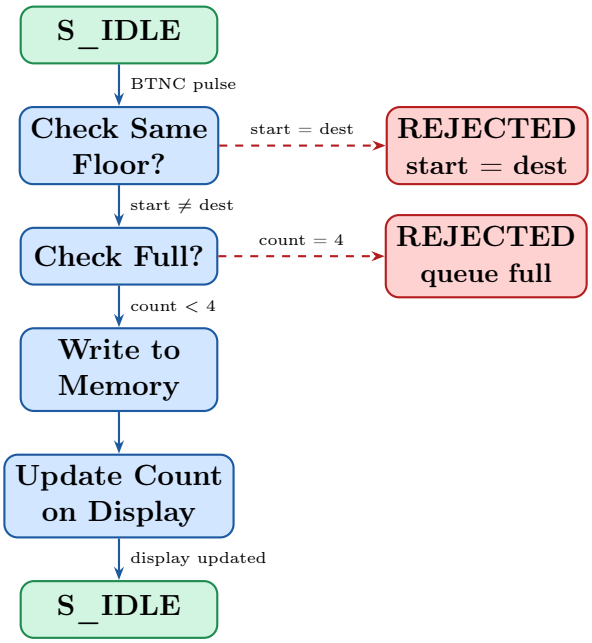

Figure 2: Passenger entry workflow. Dashed red arrows indicate rejection conditions enforced by the memory unit.

3.3 Sweep Processing and Payment Workflow

- The user presses BTNU to begin processing. The FSM latches all passenger data from memory, determines the sweep direction from the first passenger (dest $\geq$ start $\Rightarrow$ sweep up), and computes the minimum (or maximum) start floor as the sweep origin.
- The elevator moves to the first start floor, picking up passengers whose start floor matches the current floor. It then sweeps floor-by-floor in one direction.
- At each floor, the FSM checks all four passenger slots in parallel. If a passenger's destination matches the current floor:
  1. The display pauses for approximately 2 seconds showing the current floor number.

2. The display switches to fare mode, showing $|\text{dest} - \text{start}| \times 5$ taka.
3. The user sets payment on `SW[15:10]` and presses BTND. If payment $\geq$ fare, the passenger is marked delivered. Otherwise the system stays in the payment state.

- After all passengers are delivered, the display shows "EE". Pressing BTNU returns the system to IDLE for a new batch.

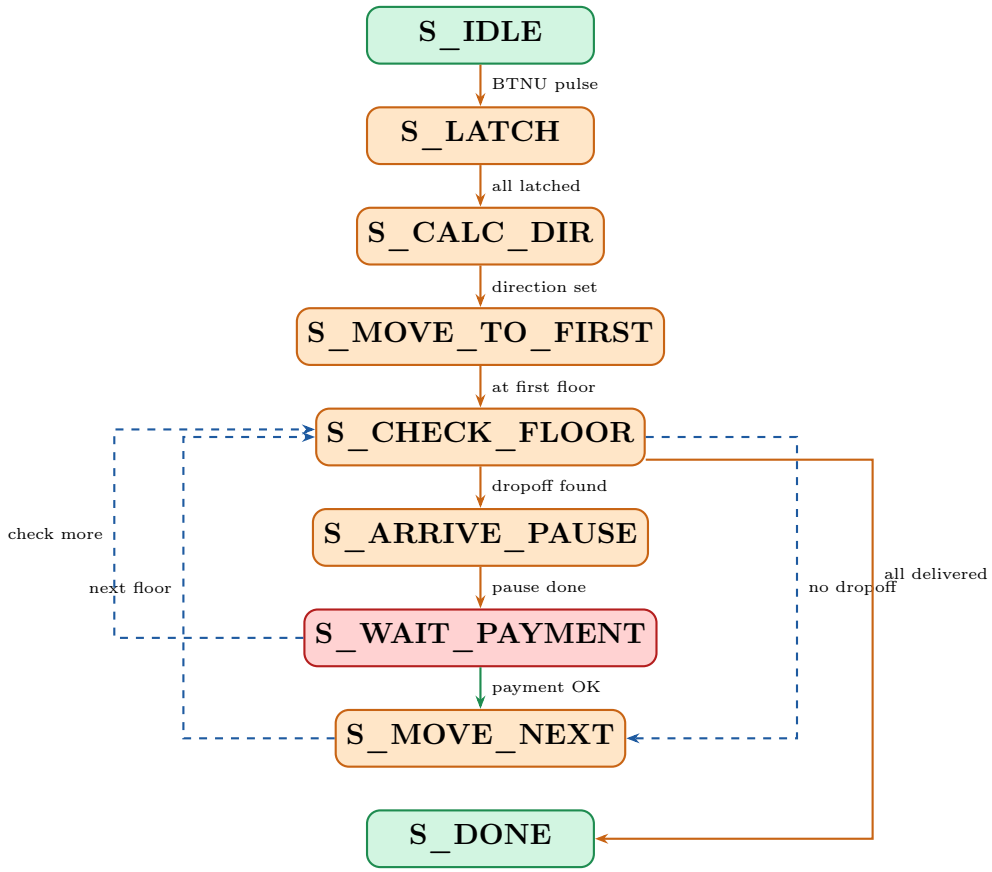

Figure 3: Sweep processing and payment workflow. Blue dashed arrows indicate the sweep loop and re-check paths. The green arrow marks the payment confirmation transition.

4 System Architecture

4.1 Detailed Architecture Diagram

The following figures break the complete system architecture into focused views. Figure 4 gives a compact overview of all seven modules and their primary interconnections. The subsequent figures zoom into the input conditioning layer (Figure 5), the central control and memory layer (Figure 6), the computation layer (Figure 8), and the output layer (Figure 9).

The overall system follows a modular and layered hardware architecture, where each module performs a dedicated function while remaining tightly synchronised within a single 100 MHz clock domain. The input conditioning layer ensures reliable user interaction by converting noisy mechanical button signals into clean, single-cycle pulses. These inputs, along with the selected floor values, are processed by the central Control Unit (FSM),

which orchestrates all operations including passenger latching, sweep direction calculation, floor-by-floor movement, pickup/dropoff detection, and fare collection. The FSM interacts with the Passenger Memory for storage of start and destination floors, with the Clock Divider for movement timing, and with the ALU for computing fare amounts. Finally, the processed results are forwarded to the output layer, where the seven-segment controller presents real-time floor numbers and fare values, while LEDs provide direction and capacity indicators.

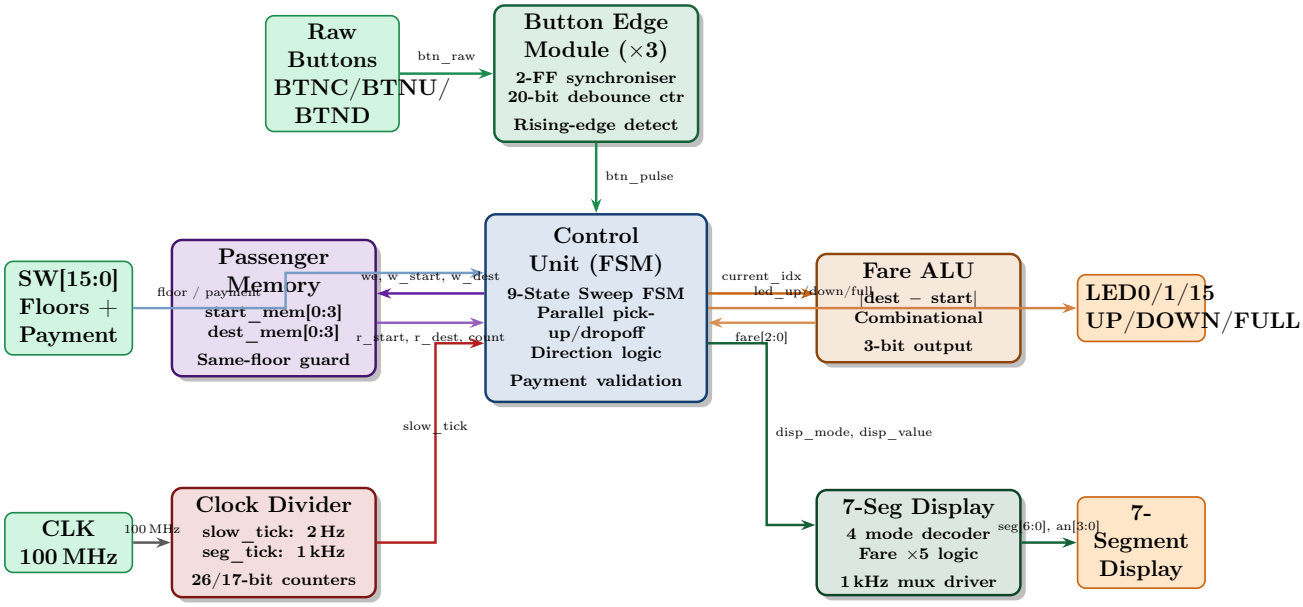

Figure 4: Detailed System Architecture Overview — All seven hardware modules and primary signal connections.

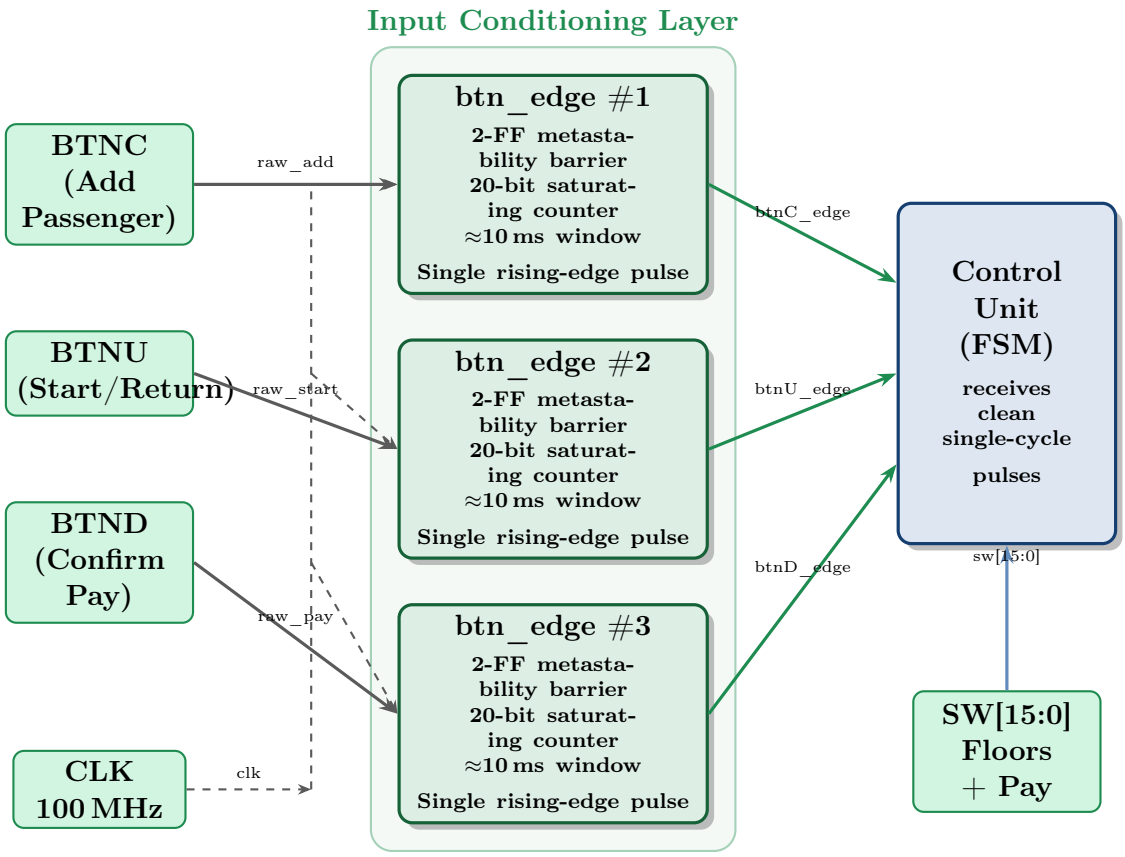

Figure 5: Input Conditioning Layer — Three independent `btn_edge` instances (one per button) and the switch path to the FSM.

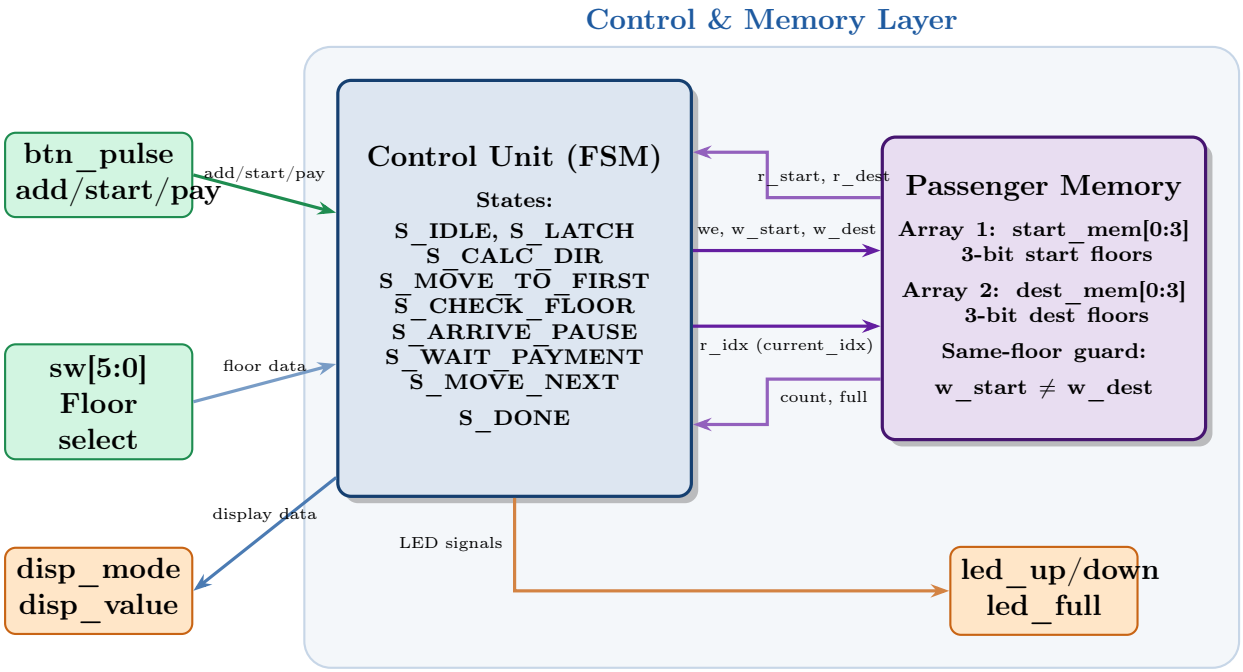

Figure 6: Control and Memory Layer — The 9-state sweep FSM together with the passenger memory unit.

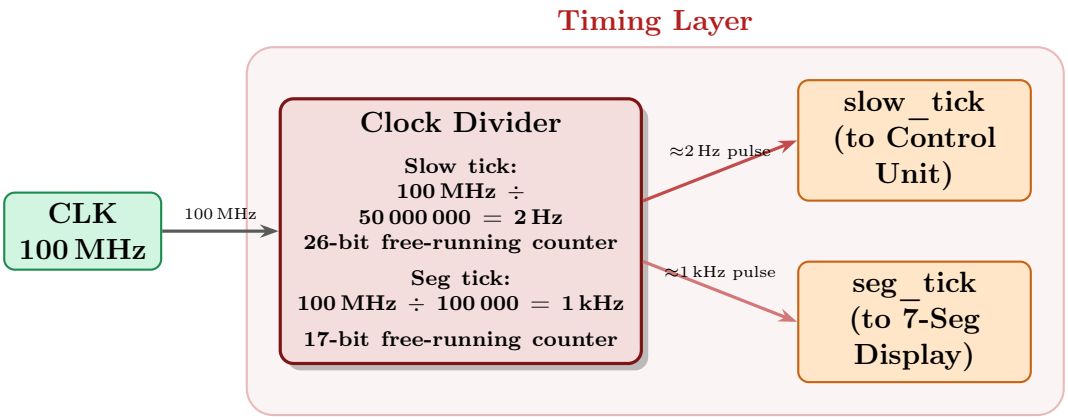

Figure 7: Timing Layer — The clock divider generates a 2 Hz movement tick for floor-by-floor elevator transitions and a 1 kHz tick for flicker-free seven-segment display multiplexing.

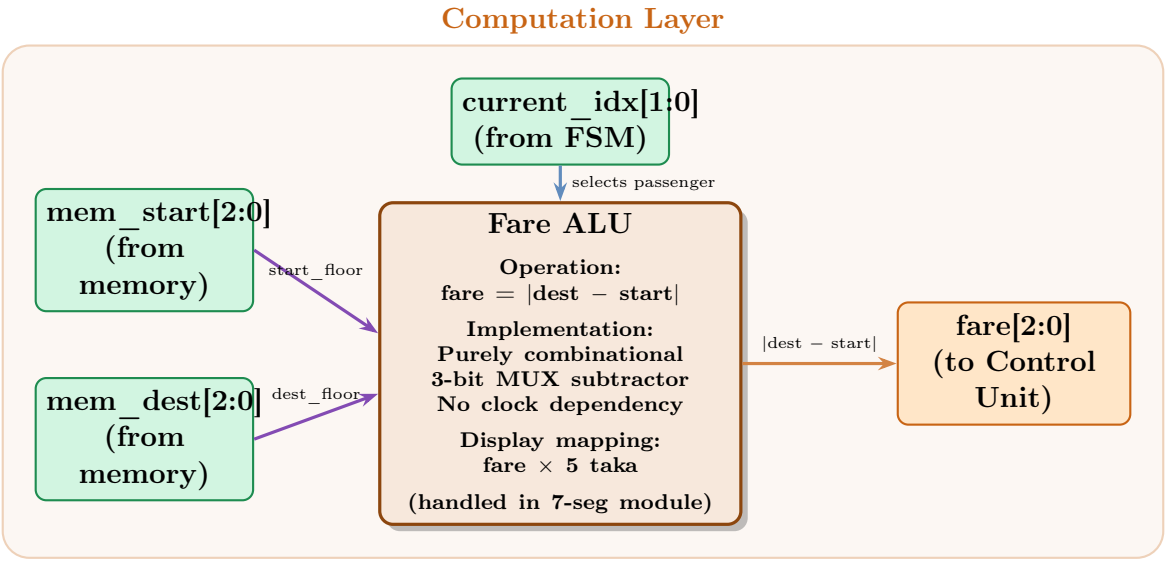

Figure 8: Computation Layer — The purely combinational fare ALU computes $|\text{dest} - \text{start}|$ for the passenger selected by `current_idx`. It has no connection to the clock divider.

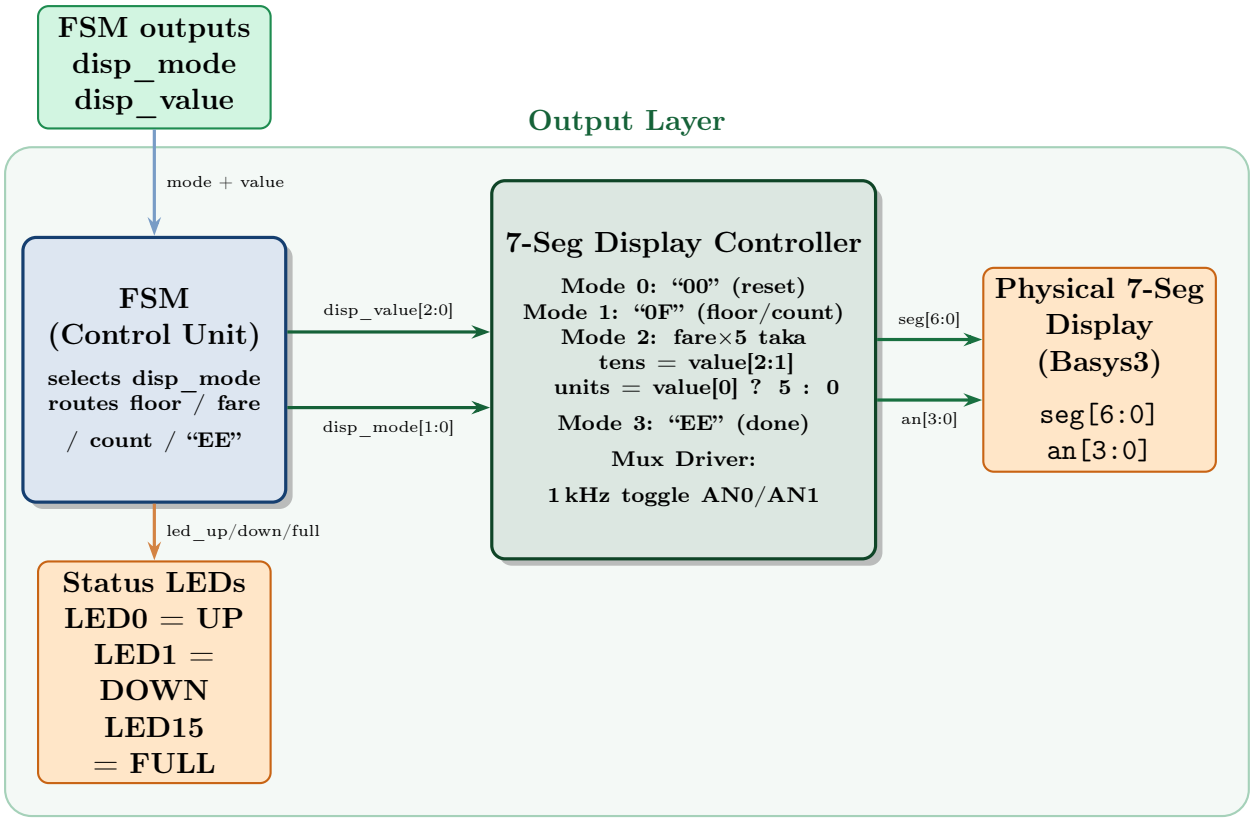

Figure 9: Output Layer — The four-mode seven-segment display driver with fare×5 conversion logic and multiplexed anode scanning, plus the three status LEDs.

4.2 Module Hierarchy

The top-level module `elevator_top` instantiates seven submodules and routes signals between them via explicit wire declarations. There is no implicit connectivity; every inter-module signal is named and typed at the top level, making the data flow self-documenting and easing simulation. The module hierarchy is as follows:

- `elevator_top` *(top level)*
  - `btn_edge` ×3 — one instance per button: `u_btn_add` (BTNC), `u_btn_start` (BTNU), `u_btn_pay` (BTND)
  - `clk_divider` — generates 2 Hz slow tick and 1 kHz segment tick from 100 MHz system clock
  - `memory_unit` — four-entry FIFO storing start and destination floors per passenger
  - `alu` — combinational absolute-difference fare calculator
  - `control_unit` — nine-state sweep-based FSM with parallel pickup/dropoff detection
  - `seven_segment` — two-digit multiplexed display driver with four display modes

4.3 RTL Schematic

Figure 10 shows the elaborated RTL schematic generated by Xilinx Vivado after synthesis of `elevator_top.v`. The schematic confirms the seven-module hierarchy and their inter-module signal routing.

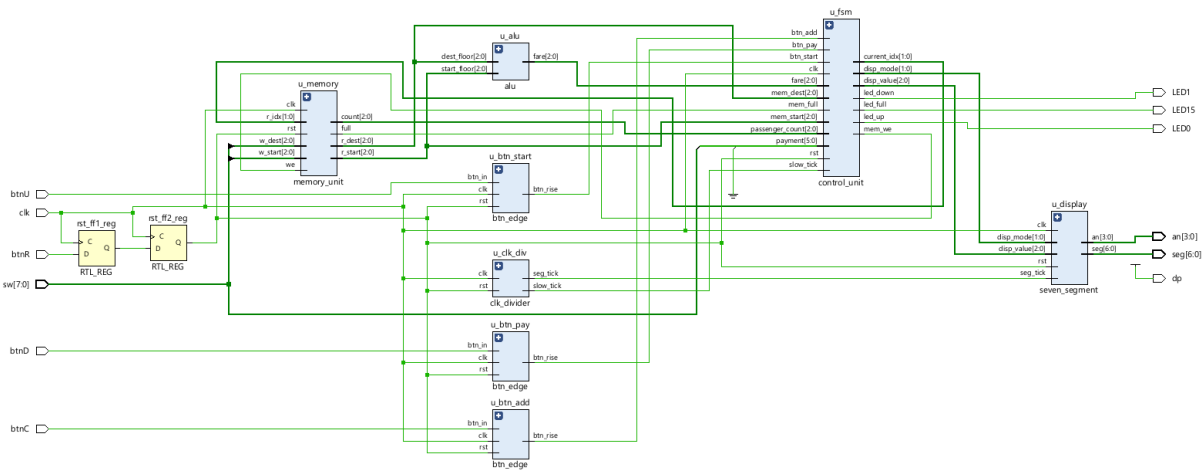

Figure 10: Elaborated RTL schematic of `elevator_top` generated by Xilinx Vivado, showing all seven submodule instances.

5 Data Flow

Figure 11 illustrates how data moves between all major subsystems. Solid arrows represent write or command signals; dashed arrows represent read-back or result signals. All flows are synchronised to the single 100 MHz clock domain.

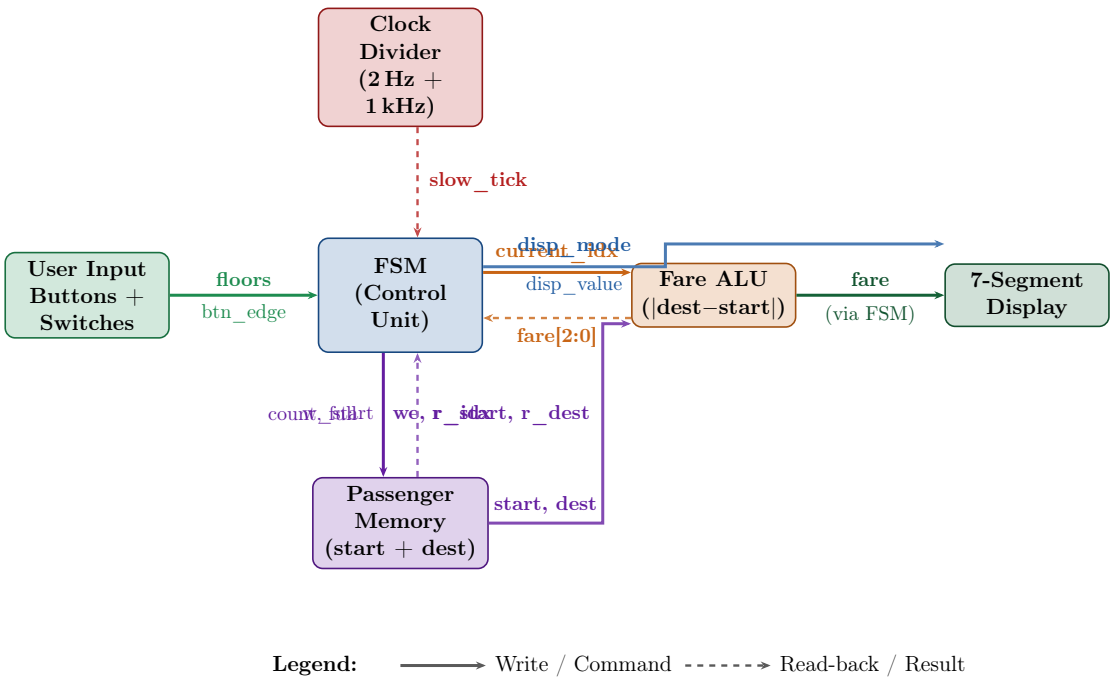

Figure 11: Data Flow Diagram — FPGA Smart Elevator Control System

5.1 Key Signal Descriptions

Table 1 summarises the principal data signals visible in the data flow diagram.

Table 1: Key data-flow signals with source, destination, and description.

Signal	Source	Destination	Description
`sw[5:3]`	Switches	Memory	3-bit start floor selected by user (0–7)
`sw[2:0]`	Switches	Memory	3-bit destination floor selected by user (0–7)
`sw[15:10]`	Switches	FSM	6-bit payment value for fare confirmation
`btn_edge`	btn_edge ×3	FSM	Single-cycle debounced add, start, and payment triggers
`slow_tick`	clk_divider	FSM	2 Hz enable pulse for floor-by-floor elevator movement
`seg_tick`	clk_divider	seven_segment	1 kHz enable pulse for display digit multiplexing
`fare`	ALU	FSM / Display	Absolute floor difference $\lvert \text{dest} - \text{start} \rvert$ (0–7)
`disp_mode`	FSM	seven_segment	Selects display mode: count, floor, fare, or "EE"
`disp_value`	FSM	seven_segment	3-bit value to display (floor number or fare floors)
`current_idx`	FSM	Memory	Read index selecting which passenger's data is output
`mem_we`	FSM	Memory	Write enable pulse to store a new passenger

6 Module Descriptions

6.1 Control Unit (FSM) — `control_unit.v`

The `control_unit` module implements a nine-state sweep-based FSM that controls all elevator operations including passenger latching, direction calculation, floor-by-floor movement, parallel pickup/dropoff detection, and fare collection. It synchronously updates all states and outputs using `always @(posedge clk)`, ensuring glitch-free behaviour.

The FSM employs four parallel combinational comparators per passenger slot to detect pickups and dropoffs at each floor simultaneously, enabling efficient multi-passenger handling without sequential scanning.

FSM States Overview:

- **S_IDLE (0):** Accepts new passengers via BTNC; starts processing via BTNU when at least one passenger is queued.

- **Initialisation Path:**
  - `S_LATCH (1):` Reads all passenger data from the memory unit into local registers (one passenger per clock cycle).
  - `S_CALC_DIR (2):` Determines sweep direction from the first passenger (dest $\geq$ start $\Rightarrow$ up) and computes the sweep origin as the minimum (or maximum) start floor.
- **Sweep Path:**
  - `S_MOVE_TO_FIRST (3):` Moves the elevator to the first start floor while displaying the target floor statically (hiding internal movement from the user).
  - `S_CHECK_FLOOR (4):` At each floor, auto-picks up passengers whose start floor matches, and checks if any picked-up passenger's destination matches for dropoff.
  - `S_ARRIVE_PAUSE (5):` Pauses for approximately 2 seconds showing the current floor before switching to fare display.
  - `S_WAIT_PAYMENT (6):` Displays the fare in taka and waits for sufficient payment (SW[15:10] $\geq$ fare).
  - `S_MOVE_NEXT (7):` Advances one floor in the sweep direction. Reverses direction at floor 0 or 7 boundaries.
- **Completion:**
  - `S_DONE (8):` Displays "EE" on the seven-segment display. Pressing BTNU returns to IDLE for a new batch.

Key Design Features:

- **Parallel Pickup/Dropoff Detection:** Four combinational comparators check all passenger slots simultaneously at each floor, using `picked_up[3:0]` and `delivered[3:0]` bitmasks.
- **Priority Encoder:** When multiple passengers exit at the same floor, the lowest-index passenger is served first.
- **Direction Auto-Reversal:** If the elevator reaches floor 0 or 7 without delivering all passengers (mixed-direction case), the sweep direction automatically reverses.
- **Min/Max Start Floor:** Combinational tree comparators compute the minimum start floor (for up-sweep) or maximum start floor (for down-sweep), ensuring all passengers are reachable during the sweep.

6.2 Fare ALU — `alu.v`

The `alu` module is a purely combinational absolute-difference calculator. It computes fare = |dest_floor − start_floor| using a ternary MUX and 3-bit subtractor:

```
assign fare = (dest_floor >= start_floor)
              ? (dest_floor - start_floor)
              : (start_floor - dest_floor);
```

Listing 1: Fare ALU implementation

The resulting 3-bit `fare` value (0–7) represents the number of floors travelled. The seven-segment display module multiplies this by 5 to show the fare in taka (e.g. fare = 3 displays as "15" taka) using the bit-trick tens = fare[2:1], units = fare[0] ? 5 : 0.

6.3 Passenger Memory — `memory_unit.v`

The memory module maintains up to four passenger records using two parallel register arrays: `start_mem[0:3]` and `dest_mem[0:3]`. It supports FIFO write (via a 3-bit `passenger_count` pointer) and random-access read (via `r_idx`).

Write Guards:

- **Same-floor rejection:** `w_start` $\neq$ `w_dest` is enforced in hardware; bookings with identical start and destination are silently discarded.
- **Overflow protection:** Writes are blocked when `passenger_count` = 4.

```
always @(posedge clk) begin
    if (rst)
        passenger_count <= 3'd0;
    else if (we && !full && (w_start != w_dest)) begin
        start_mem[passenger_count[1:0]] <= w_start;
        dest_mem[passenger_count[1:0]]  <= w_dest;
        passenger_count <= passenger_count + 3'd1;
    end
end
```

Listing 2: Memory write guard logic

6.4 Clock Divider — `clk_divider.v`

The `clk_divider` module generates two enable pulses from the 100 MHz system clock:

- **slow_tick (≈2 Hz):** 100 MHz ÷ 50,000,000 = 2 Hz. Used for floor-by-floor elevator movement. Each floor transition takes approximately 0.5 seconds.
- **seg_tick (≈1 kHz):** 100 MHz ÷ 100,000 = 1 kHz. Used for flicker-free multiplexing of the two-digit seven-segment display.

Both ticks are single-cycle high pulses, implemented using free-running counters with terminal-count comparison.

6.5 Seven-Segment Display — `seven_segment.v`

The display controller multiplexes two digits (AN0 and AN1) at 1 kHz and supports four display modes:

Table 2: Seven-segment display modes.

Mode	Value	AN1 (tens)	AN0 (units)
0	—	0	0
1	floor/count	0	`disp_value`
2	fare floors	value[2:1]	value[0] ? 5 : 0
3	—	E	E

The fare display uses a hardware-efficient multiplication-by-5 decomposition: for a fare of n floors, the tens digit is $\lfloor n/2 \rfloor$ and the units digit is $(n \bmod 2) \times 5$, yielding correct decimal values 05, 10, 15, 20, 25, 30, 35 for $n = 1 \ldots 7$.

6.6 Button Edge Detector — `btn_edge.v`

The `btn_edge` module provides reliable mechanical button input through a three-stage pipeline:

1. **2-FF Synchroniser:** Eliminates metastability from the asynchronous button input crossing into the 100 MHz clock domain.
2. **Counter-Based Debouncer:** A 20-bit saturating counter (`DEBOUNCE_CYCLES` = 1,000,000 $\approx$ 10 ms) rejects transitions shorter than the debounce window.
3. **Rising-Edge Detector:** Produces a guaranteed single-cycle pulse `btn_rise` on the debounced low-to-high transition.

7 Implementation Details

7.1 Basys3 FPGA Specifications

The Basys3 development board hosts a Xilinx Artix-7 FPGA (XC7A35T-1CPG236C). The project utilises a modest fraction of available resources as summarised in Table 3. All arrays are implemented as distributed RAM; no block RAM tiles or DSP slices are consumed.

Table 3: Approximate FPGA resource utilisation.

Resource	Used	Available
LUT6 equivalents	~250	20,800
Flip-Flops	~180	41,600
DSP48E1 slices	0	90
Block RAM (36 Kb)	0	50
IOBs	38	106

7.2 Input/Output Pin Mapping

Table 4: Complete I/O pin mapping as defined in `Basys3_Elevator.xdc`.

Signal	Pin / Switch	Dir.	Description
`clk`	W5	In	100 MHz system clock oscillator
`btnR`	T17	In	Synchronous reset (active HIGH)
`btnC`	U18	In	Add passenger to queue (debounced)
`btnU`	T18	In	Start processing / return from DONE
`btnD`	U17	In	Confirm payment
`sw[2:0]`	V17–W16	In	3-bit destination floor select (0–7)
`sw[5:3]`	W17–V15	In	3-bit start floor select (0–7)
`sw[15:10]`	R2–T2	In	6-bit payment value (upper switches)
`LED0`	U16	Out	Moving UP indicator
`LED1`	E19	Out	Moving DOWN indicator
`LED15`	L1	Out	Memory FULL indicator
`seg[6:0]`	W7–U7	Out	7-segment cathodes (active-low)
`an[3:0]`	U2–W4	Out	7-segment anodes (active-low)
`dp`	V7	Out	Decimal point (permanently OFF)

7.3 Fare Calculation

The fare is computed as:

$$\text{fare (taka)} = |\text{destination floor} - \text{start floor}| \times 5$$

The ALU computes the absolute difference combinationally, and the seven-segment module performs the $\times 5$ conversion using bit manipulation rather than a hardware multiplier, conserving DSP resources.

Table 5: Fare lookup table for all possible floor differences.

Floors Travelled	Fare (taka)	Display
1	5	05
2	10	10
3	15	15
4	20	20
5	25	25
6	30	30
7	35	35

8 Testing and Verification

Functional correctness was established through on-board hardware validation on the Basys3. Table 6 summarises the test cases executed and their results.

Table 6: Hardware test cases and results.

#	Test Case	Description	Result
1	Power-On Reset	All outputs initialise correctly after BTNR	PASS
2	Add Passenger	BTNC queues passenger, count increments to 01	PASS
3	Full Ride UP	Floor 0→3, fare shows 15, payment accepted	PASS
4	Full Ride DOWN	Floor 5→1, LED1 ON, fare shows 20	PASS
5	Multi-Passenger Sweep	2 passengers (same start, different dests) processed in one sweep	PASS
6	Memory Full	Count stops at 4, LED15 ON, 5th passenger rejected	PASS
7	Same-Floor Rejection	Start = dest silently rejected	PASS
8	Insufficient Payment	Payment < fare rejected, stays on fare display	PASS
9	Max Distance (0→7)	All 7 transitions visible, fare = 35	PASS

9 Results and Discussion

The synthesised design achieves timing closure at 100 MHz with no setup or hold violations on any path, owing to the absence of pipelined multipliers or deep combinational chains. Resource utilisation is minimal: approximately 1.2% of available LUTs and 0.4% of flip-flops, with zero DSP slices and zero block RAM tiles consumed, as the fare ALU is purely combinational and all memory arrays fit within distributed LUTRAM.

The sweep-based FSM architecture represents a significant improvement over the initial sequential per-passenger design. In the original implementation, the elevator would return to each passenger's start floor individually, resulting in redundant travel. The sweep approach picks up all passengers at their start floors during a single directional pass and drops them off at their destinations in floor order—exactly matching real-world elevator behaviour. This reduces the total floor transitions from $O(2n \times \text{avg_distance})$ to $O(\text{max_dest} - \text{min_start})$ for n co-directional passengers.

The four-mode seven-segment display provides clear visual feedback throughout all FSM states. The fare×5 conversion using bit-decomposition (tens = value[2:1], units = value[0]?5 : 0) avoids a hardware multiplier while producing correct decimal output for all seven fare values. The 2-second arrival pause provides the user adequate time to observe the destination floor before the fare appears.

Known limitations include: the 3-bit floor encoding restricts the building to eight floors (0–7); the four-entry memory limits the passenger batch size; all records reside in volatile storage and are lost on reset; and the sweep direction is determined solely from the first

passenger, which may not be optimal for mixed-direction batches (although auto-reversal at boundaries handles this case correctly, albeit with suboptimal travel distance).

10 Conclusion

This project has demonstrated the complete design, implementation, and hardware validation of a sweep-based multi-passenger elevator control system on the Digilent Basys3, described in synthesisable Verilog-2001 and targeting a Xilinx Artix-7 (XC7A35T-1CPG236C). The system integrates seven hardware modules within a single 100 MHz clock domain: a nine-state sweep-based FSM with parallel pickup/dropoff detection via four combinational comparators per floor; a combinational absolute-difference fare ALU; a four-entry FIFO passenger memory with same-floor rejection and overflow protection; a dual-rate clock divider producing 2 Hz movement ticks and 1 kHz display ticks; a four-mode multiplexed seven-segment display driver with hardware fare×5 conversion; and three synchroniser-debounce-edge-detector instances providing 10 ms bounce rejection.

All nine test cases passed on physical hardware, confirming correct FSM transitions across all operating modes, accurate fare computation and display for all seven fare values (5–35 taka), reliable button debouncing under repeated actuation, proper same-floor rejection, memory-full enforcement, and correct sweep behaviour with multiple passengers sharing start floors but having different destinations. The final design meets all timing requirements and occupies negligible device resources, demonstrating that a functionally complete real-time elevator control system can be specified, synthesised, and physically validated through rigorous application of synchronous design principles and modular architecture.

11 Future Work

Several extensions would increase the practical applicability of this system:

- **Extended Floor Range:** Widening the floor encoding from 3 to 4 or 5 bits would support 16- or 32-storey buildings with minimal RTL changes.
- **Larger Passenger Queue:** Expanding the memory from 4 to 8 or 16 entries using block RAM would accommodate higher-traffic scenarios.
- **Multi-Direction Optimisation:** Implementing a look-ahead scheduler that partitions passengers into up-bound and down-bound groups before sweeping would reduce total travel distance for mixed-direction batches.
- **UART Logging:** Adding a UART transmitter over the Basys3 USB-UART bridge would enable real-time transaction logging (passenger ID, floor, fare, timestamp) to a host PC.
- **Variable Fare Rate:** Replacing the fixed ×5 fare multiplier with a switch-configurable rate would allow dynamic pricing.
- **Formal Verification:** Applying property checking via SymbiYosys to `control_unit.v` would provide machine-verified proofs of FSM deadlock-freedom and delivery-liveness guarantees that simulation alone cannot establish.

References

[1] Digilent Inc., "Basys3 Reference Manual," Revision E, 2019. [Online]. Available: https://digilent.com/reference/programmable-logic/basys-3/reference-manual

[2] Xilinx Inc., "7 Series FPGAs Data Sheet: Overview," DS180, v2.6, 2020.

[3] Xilinx Inc., "Vivado Design Suite User Guide: Synthesis," UG901, v2023.1, 2023.

[4] S. Palnitkar, *Verilog HDL: A Guide to Digital Design and Synthesis*, 2nd ed. Upper Saddle River, NJ: Prentice Hall, 2003.

[5] M. D. Ciletti, *Advanced Digital Design with the Verilog HDL*, 2nd ed. Upper Saddle River, NJ: Pearson Prentice Hall, 2011.

[6] P. P. Chu, *FPGA Prototyping by Verilog Examples: Xilinx Spartan-3 Version*. Hoboken, NJ: Wiley-IEEE Press, 2008.

[7] IEEE, *IEEE Standard for Verilog Hardware Description Language*, IEEE Std 1364-2001, New York, NY: IEEE, 2001.

[8] Xilinx Inc., "Basys3 Master XDC Constraint File," Digilent Vivado Scripts, GitHub, 2022. [Online]. Available: https://github.com/Digilent/digilent-xdc